

# EXOHUB MATRIX

## ASTRONOMY CLUB SUMMER PROJECT' 25

---

### Mentors

Rishabh Kumar  
Rishabh Verma  
Toshika Kamble  
Mayank Pattnaik

### Mentees

|                 |                    |
|-----------------|--------------------|
| Tanmay Shukla   | Aman Pal           |
| Astha Yadav     | Abhinav Bhardwaj   |
| Dalima          | Harshit Pandey     |
| Haryashva Gupta | Ishaan Dasgupta    |
| Mahi Mittal     | Lalit Meena        |
| Mayank Agrawal  | Mihir Tejaswi      |
| Mishthi Sharma  | Nikhil Patel       |
| Saket Garg      | Pranjali Deshpande |
| Tripti Kushwaha | Mahi Mittal        |

# Contents

|          |                                                                                  |          |
|----------|----------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Atmospheric Composition Spectroscopy</b>                                      | <b>3</b> |
| 1.1      | Introduction . . . . .                                                           | 3        |
| 1.2      | Principles of Spectroscopy in Atmospheric Analysis . . . . .                     | 3        |
| 1.2.1    | Spectroscopic Fingerprints . . . . .                                             | 3        |
| 1.2.2    | Types of Atmospheric Spectroscopy . . . . .                                      | 3        |
| 1.3      | Mathematical Foundations . . . . .                                               | 4        |
| 1.3.1    | The Rydberg Equation . . . . .                                                   | 4        |
| 1.3.2    | Beer-Lambert Law . . . . .                                                       | 4        |
| 1.3.3    | Inverse Retrieval Techniques . . . . .                                           | 4        |
| 1.4      | Measurement Techniques and Instrumentation . . . . .                             | 5        |
| 1.4.1    | Fourier Transform Infrared (FTIR) Spectroscopy . . . . .                         | 5        |
| 1.4.2    | Laser-Induced Breakdown Spectroscopy (LIBS) . . . . .                            | 5        |
| 1.4.3    | Atomic Absorption Spectroscopy (AAS) . . . . .                                   | 5        |
| 1.5      | Example Application: Multiscale Spectral Analysis of Atmospheric Gases . . . . . | 5        |
| 1.5.1    | Objective . . . . .                                                              | 5        |
| 1.5.2    | Methodology . . . . .                                                            | 5        |
| 1.5.3    | Results (Hypothetical Example) . . . . .                                         | 6        |
| 1.6      | Conclusion . . . . .                                                             | 6        |
| <b>2</b> | <b>Spectroscopy Analysis Using Python</b>                                        | <b>7</b> |
| <b>3</b> | <b>Basics of Machine Learning</b>                                                | <b>9</b> |
| 3.1      | Intro to ML . . . . .                                                            | 9        |
| 3.1.1    | Basics of ML . . . . .                                                           | 9        |
| 3.1.2    | Data Exploration/Analysis . . . . .                                              | 9        |
| 3.1.3    | Data Preprocessing and Augmentation . . . . .                                    | 9        |
| 3.1.4    | Train-Test Split . . . . .                                                       | 9        |
| 3.1.5    | Model Architecture . . . . .                                                     | 9        |
| 3.1.6    | Model Training . . . . .                                                         | 9        |
| 3.1.7    | Hyperparameters . . . . .                                                        | 9        |
| 3.1.8    | Error, Accuracy, and Other Metrics . . . . .                                     | 10       |
| 3.2      | Random Forest Regression . . . . .                                               | 10       |
| 3.2.1    | Introduction to Random Forests . . . . .                                         | 10       |
| 3.2.2    | When to Use Random Forest Regression . . . . .                                   | 10       |
| 3.3      | Support Vector Machines (SVM) . . . . .                                          | 10       |
| 3.3.1    | Introduction to SVM . . . . .                                                    | 10       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 3.3.2    | When to Use SVM . . . . .                                       | 11        |
| 3.4      | CNN for Facial Expression Recognition . . . . .                 | 11        |
| 3.4.1    | Introduction to CNNs . . . . .                                  | 11        |
| 3.5      | Data Exploration and Analysis . . . . .                         | 11        |
| <b>4</b> | <b>Data Preprocessing and Augmentation for CNN Models</b>       | <b>13</b> |
| 4.1      | Introduction . . . . .                                          | 13        |
| 4.2      | General Preprocessing Steps . . . . .                           | 13        |
| 4.2.1    | Why is it needed? . . . . .                                     | 13        |
| 4.2.2    | Defining Data Transforms . . . . .                              | 14        |
| 4.3      | Data Augmentation for Face Recognition . . . . .                | 14        |
| 4.3.1    | Augmentation Techniques . . . . .                               | 15        |
| 4.3.2    | Creating Data Loaders . . . . .                                 | 15        |
| 4.4      | Conclusion . . . . .                                            | 16        |
| <b>5</b> | <b>CNN Architecture for Facial Expression Detection</b>         | <b>17</b> |
| 5.1      | Introduction to Convolutional Neural Networks (CNNs) . . . . .  | 17        |
| 5.1.1    | Core Layers of a CNN . . . . .                                  | 17        |
| 5.2      | Our CNN Model Architecture . . . . .                            | 18        |
| 5.2.1    | Hyperparameters . . . . .                                       | 18        |
| 5.2.2    | Benefits of this Architecture . . . . .                         | 19        |
| 5.3      | Implementation in TensorFlow . . . . .                          | 19        |
| 5.3.1    | Code Snippets . . . . .                                         | 19        |
| 5.4      | Parameter Count and Distribution . . . . .                      | 20        |
| <b>6</b> | <b>Facial Emotion Recognition Using CNN</b>                     | <b>22</b> |
| 6.1      | Introduction . . . . .                                          | 22        |
| 6.2      | 1. Data Analysis and Format . . . . .                           | 22        |
| 6.3      | 2. Data Preparation: Splitting and Loading . . . . .            | 22        |
| 6.4      | 3. Building the CNN Model . . . . .                             | 23        |
| 6.5      | 4. Model Training . . . . .                                     | 23        |
| 6.6      | 5. Model Evaluation . . . . .                                   | 24        |
| 6.7      | 6. Testing on Custom Images . . . . .                           | 24        |
| 6.8      | Summary . . . . .                                               | 24        |
| <b>7</b> | <b>Ranking of Planets</b>                                       | <b>25</b> |
| 7.1      | ESI Calculation – Datasets, Models, and Final Results . . . . . | 25        |
| 7.1.1    | Objective . . . . .                                             | 25        |
| 7.1.2    | Datasets Used . . . . .                                         | 25        |
| 7.2      | Modeling Techniques . . . . .                                   | 26        |
| 7.2.1    | Random Forest Regressor (Pipeline 1) . . . . .                  | 26        |
| 7.2.2    | Support Vector Regressor (Pipeline 2) . . . . .                 | 26        |
| 7.3      | Application to External Datasets . . . . .                      | 27        |
| 7.3.1    | Prediction on Kepler Data . . . . .                             | 27        |
| 7.3.2    | Prediction on TESS Data . . . . .                               | 27        |
| 7.4      | Conclusion . . . . .                                            | 28        |

# Atmospheric Composition Spectroscopy

## 1.1. Introduction

Atmospheric composition spectroscopy is a scientific discipline focused on identifying and quantifying the chemical species present in the Earth's atmosphere by analyzing their interaction with electromagnetic radiation. This field leverages the unique spectral signatures of atoms and molecules to reveal detailed information about the atmosphere's makeup, structure, and dynamic processes. The approach is foundational in atmospheric science, remote sensing, and planetary exploration.

## 1.2. Principles of Spectroscopy in Atmospheric Analysis

### 1.2.1. Spectroscopic Fingerprints

Every atom and molecule has a unique set of energy levels. When electromagnetic radiation passes through or is emitted by a sample, specific wavelengths are absorbed or emitted due to transitions between these energy levels. This produces a characteristic spectrum—a “fingerprint”—for each species.

- **Atomic Absorption and Emis-**

**sion:** Atoms absorb or emit photons at discrete wavelengths, determined by their electronic structure. For example, sodium atoms emit a bright yellow doublet at 589 nm, while oxygen and nitrogen have distinct bands in the UV and visible regions.

- **Molecular Spectra:** Molecules exhibit more complex spectra due to rotational and vibrational transitions, especially prominent in the infrared region.

### 1.2.2. Types of Atmospheric Spectroscopy

- **Atomic Absorption Spectroscopy (AAS):** Measures the absorption of light by free atoms, commonly used for detecting metal species.
- **Atomic Emission Spectroscopy (AES):** Analyzes light emitted by excited atoms, often after excitation by a flame or plasma.
- **Fourier Transform Infrared (FTIR) Spectroscopy:** Captures broad spectral information, enabling simultaneous detection of multiple

gases based on their IR absorption features.

- **Laser-Induced Breakdown Spectroscopy (LIBS):** Uses a focused laser pulse to generate plasma from atmospheric particles, with the emitted light revealing elemental composition.
- **Remote Sensing Spectroscopy:** Satellite-borne instruments measure the absorption or emission of atmospheric constituents over large spatial scales.

### 1.3. Mathematical Foundations

#### 1.3.1. The Rydberg Equation

The Rydberg equation is fundamental in atomic spectroscopy, describing the wavelengths of photons absorbed or emitted as electrons transition between energy levels in hydrogen-like atoms:

$$\tilde{\nu} = R_H \left( \frac{1}{n_1^2} - \frac{1}{n_2^2} \right) \quad (1.1)$$

where

- $\tilde{\nu}$  = Wavenumber of the spectral line ( $\text{cm}^{-1}$ )
- $R_H$  = Rydberg constant ( $109,737 \text{ cm}^{-1}$ )
- $n_1, n_2$  = Principal quantum numbers ( $n_2 > n_1$ )

This equation helps predict the positions of spectral lines, which are then used to identify elements in the atmosphere.

#### 1.3.2. Beer-Lambert Law

The Beer-Lambert Law quantifies how much light is absorbed as it travels through a medium containing an absorbing species:

$$I = I_0 e^{-\sigma N l} \quad (1.2)$$

where

- $I_0$  = Incident light intensity
- $I$  = Transmitted intensity
- $\sigma$  = Absorption cross-section ( $\text{cm}^2/\text{molecule}$ )
- $N$  = Number density of absorbers ( $\text{molecules}/\text{cm}^3$ )
- $l$  = Path length (cm)

This law is foundational for quantitative spectroscopic analysis of atmospheric gases.

#### 1.3.3. Inverse Retrieval Techniques

Modern atmospheric spectroscopy, especially from satellites, relies on inverse methods to extract gas concentrations from measured spectra. These methods use:

- **Forward models:** Simulate expected spectra for given atmospheric conditions.
- **A priori information:** Statistical assumptions about atmospheric states.
- **Averaging kernels:** Describe the sensitivity and resolution of the retrieval at different altitudes.

Table 1.1: Common Atmospheric Spectroscopy Techniques

| Technique                | Principle                        | Typical Use Cases                                      |
|--------------------------|----------------------------------|--------------------------------------------------------|
| FTIR Spectroscopy        | IR absorption by molecules       | Multi-gas detection, profiling                         |
| UV-Vis Spectroscopy      | UV/visible absorption/emission   | Ozone, NO <sub>2</sub> , SO <sub>2</sub> , trace gases |
| LIBS                     | Laser-induced plasma emission    | Elemental analysis of aerosols                         |
| Satellite Remote Sensing | Nadir or limb viewing of spectra | Global mapping of composition                          |

## 1.4. Measurement Techniques and Instrumentation

### 1.4.1. Fourier Transform Infrared (FTIR) Spectroscopy

FTIR instruments, such as the Infrared Atmospheric Sounding Interferometer (IASI), provide global, high-resolution measurements of atmospheric gases by capturing infrared spectra from space. The technique allows for the retrieval of vertical profiles and total columns of species like CO, CH<sub>4</sub>, O<sub>3</sub>, and NH<sub>3</sub>.

### 1.4.2. Laser-Induced Breakdown Spectroscopy (LIBS)

LIBS enables real-time, *in situ* detection of atmospheric particulate matter by generating plasma from aerosols and analyzing the emitted light for elemental composition. This is particularly useful for monitoring aerosols in various environments, including industrial and indoor settings.

### 1.4.3. Atomic Absorption Spectroscopy (AAS)

AAS is widely used for quantifying metal atoms in atmospheric samples. It operates by measuring the absorption of element-specific wavelengths by vaporized atoms in a flame or graphite furnace.

## 1.5. Example Application: Multiscale Spectral Analysis of Atmospheric Gases

Inspired by the methodology in [arXiv:2310.09352](#)

### 1.5.1. Objective

To demonstrate the use of high-resolution spectroscopy for analyzing the temporal evolution and vertical distribution of a key atmospheric gas, such as nitrogen dioxide (NO<sub>2</sub>), over an urban region.

### 1.5.2. Methodology

- **Data Acquisition:** Continuous spectral measurements are collected using a UV-Vis spectrometer with sub-nanometer resolution.
- **Spectral Decomposition:** Advanced signal processing (e.g., independent component analysis) is applied to separate overlapping absorption features and distinguish NO<sub>2</sub> from other gases and water vapor interference.

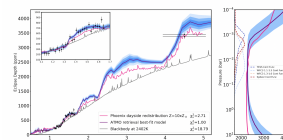


Figure 1.1: spectroscopy of KELT-20b

- **Inverse Retrieval:** The measured spectra are input into a retrieval al-

gorithm, using a priori information and averaging kernels to estimate the vertical profile of  $\text{NO}_2$  concentration.

- **Temporal Analysis:** The dataset is analyzed to extract diurnal and seasonal patterns, as well as short-term fluctuations due to meteorological changes.

### 1.5.3. Results (Hypothetical Example)

- **Spectral Identification:**  $\text{NO}_2$  absorption features are clearly resolved, even in the presence of overlapping signals from other gases.
- **Vertical Profiling:** The retrieval shows a concentration peak in the lower troposphere, consistent with expected emission sources and atmospheric mixing.

- **Temporal Trends:** Analysis reveals regular daily cycles and episodic enhancements corresponding to specific meteorological events.

## 1.6. Conclusion

Atmospheric composition spectroscopy is a cornerstone of modern atmospheric science. By leveraging the unique spectral signatures of atoms and molecules, it enables precise identification and quantification of atmospheric constituents. Techniques such as FTIR, LIBS, and AAS, combined with robust mathematical models like the Rydberg equation and Beer-Lambert law, provide a comprehensive toolkit for probing the atmosphere's structure and dynamics. The field continues to advance rapidly, driven by improvements in instrumentation, data analysis, and remote sensing capabilities.

---

# Spectroscopy Analysis Using Python

## Overview

Data taken for analysis is derived from the PEPSI Exoplanet Transit Survey (PETS) and PEPSI Investigation, Retrieval, and Atlas of Numerous Giant Atmospheres (PIRANGA) projects.

## Spectrum Preparation and Stitching

- The captured spectrum is divided into two parts: blue (4800–5541 Å) and red (6728–7419 Å) arms of the spectrograph. These are stitched together using data analysis techniques to construct a final, overall spectrum.
- The combined spectra are sorted by ascending wavelength, ensuring the spectrum is continuous.
- Optionally, in any overlapping region, the flux values at identical wavelengths are averaged (or one spectrum is interpolated onto the other's grid) to smooth the join.

## Preprocessing for Peak Detection

- The flux is inverted (i.e., subtracted from its maximum) so that absorption dips become peaks.
- `scipy.signal.find_peaks` is used on the inverted spectrum. By tuning parameters such as prominence, width, and distance, genuine atmospheric absorption lines can be picked out, noise ignored, and the detected peaks stored in an array.

## Peak Identification and Elemental Analysis

- A dictionary of known atomic or molecular peaks (from the NIST Atomic Spectra Database) is imported into the code. The detected peaks are checked against this dictionary.
- For each detected dip, its wavelength is checked to see if it lies within a chosen tolerance (typically  $\pm$  a few Å) of any known line from the dictionary, flagging likely matches.



- The entire process can be automated by changing the prominence levels and threshold values for the baseline, aiding in smooth detection of elemental peaks.
- The abundance of each element can be determined by integrating the equivalent width of each absorption feature and comparing the flux for each element against the normal baseline flux, then converting to column density.

---

# Basics of Machine Learning

## 3.1. Intro to ML

### 3.1.1. Basics of ML

Machine Learning (ML) is a subset of artificial intelligence that focuses on developing algorithms and statistical models, enabling computers to perform tasks without explicit instructions. These systems learn from data by identifying patterns and making decisions based on them.

### 3.1.2. Data Exploration/Analysis

Data exploration and analysis involve examining the dataset's characteristics, such as its size, distribution, and quality. This step helps identify issues like missing values, outliers, or class imbalances that could impact model performance.

### 3.1.3. Data Preprocessing and Augmentation

Data preprocessing entails cleaning the data, handling missing values, normalizing features, and encoding categorical variables. Augmentation techniques, such as rotation or flipping, artificially expand the dataset to enhance model generalization.

### 3.1.4. Train-Test Split

The train-test split divides the dataset into a training set, used to train the model, and a test set, used to assess its performance on unseen data. This method evaluates how well the model generalizes.

### 3.1.5. Model Architecture

Model architecture defines the structure of a machine learning model, such as the arrangement of layers in a neural network. For instance, a Convolutional Neural Network (CNN) includes convolutional, pooling, and fully connected layers.

### 3.1.6. Model Training

Model training involves feeding training data into the model and optimizing its parameters to minimize prediction errors. This process typically uses algorithms like gradient descent.

### 3.1.7. Hyperparameters

Hyperparameters are predefined settings that control the training process, such as learning rate, batch size, and the number of epochs. The number of epochs determines how many times the model will pass through the entire training dataset, with

each pass (epoch) allowing the model to iteratively refine its learning. These settings are adjusted before training begins to optimize model performance.

### 3.1.8. Error, Accuracy, and Other Metrics

Error metrics, like mean squared error or cross-entropy loss, quantify model performance. Accuracy measures the proportion of correct predictions in classification tasks, while metrics like precision, recall, and F1-score offer deeper insights.

## 3.2. Random Forest Regression

### 3.2.1. Introduction to Random Forests

It builds multiple decision trees during training and aggregates their outputs, improving accuracy and reducing overfitting compared to individual decision trees.

- **Trees:** A Random Forest constructs several decision trees, each trained on a random subset of the data and features.
- **Ensemble Learning:** The final prediction is made by averaging the outputs of all trees (regression) or using majority voting (classification).
- **Variance Reduction:** By averaging across many trees, Random Forest minimizes the risk of overfitting and improves robustness.
- **Feature Importance:** Random Forests can measure the relative importance of input features, helping interpret which factors most influence predictions.
- **Versatility:** They handle large datasets with higher dimensionality

and can manage missing values and outliers effectively.

### 3.2.2. When to Use Random Forest Regression

Random Forest Regression excels in situations with:

- Complex, non-linear relationships between input variables and the target.
- Datasets with many features, including categorical ones.
- A need for model interpretability via feature importance.

## 3.3. Support Vector Machines (SVM)

### 3.3.1. Introduction to SVM

Support Vector Machines (SVM) are supervised learning models used for classification and regression tasks. SVMs aim to find the best decision boundary (called a hyperplane) that maximizes the margin between different classes.

- **Hyperplane:** A decision boundary that separates data points of different classes in the feature space.
- **Support Vectors:** The closest data points to the hyperplane which influence its position and orientation.
- **Margin:** The distance between the hyperplane and the nearest support vectors; SVM attempts to maximize this margin for better generalization.
- **Kernel Trick:** A technique that allows SVMs to perform well with non-linearly separable data by mapping input features into higher-dimensional spaces.

- **Soft Margin:** Allows some misclassifications to enable better generalization on noisy data by introducing slack variables.

### 3.3.2. When to Use SVM

SVMs are particularly effective when:

- The data has clear margin of separation.
- High-dimensional spaces are involved.
- You need robust models that work well on small- to medium-sized datasets.
- Nonlinear classification or regression is needed, where kernel methods can be applied.

## 3.4. CNN for Facial Expression Recognition

We, for our Assignment 3, were tasked with building a Convolutional Neural Network (CNN) to recognize facial expressions. The goal is to classify face images into one of seven emotion categories.

### 3.4.1. Introduction to CNNs

Convolutional Neural Networks (CNNs) are deep learning models highly effective for image processing. Key components include:

- **Kernel:** A small matrix applied during convolution to extract features like edges or textures from the input image.
- **Convolution:** The operation of sliding the kernel over the image,

computing dot products to generate feature maps.

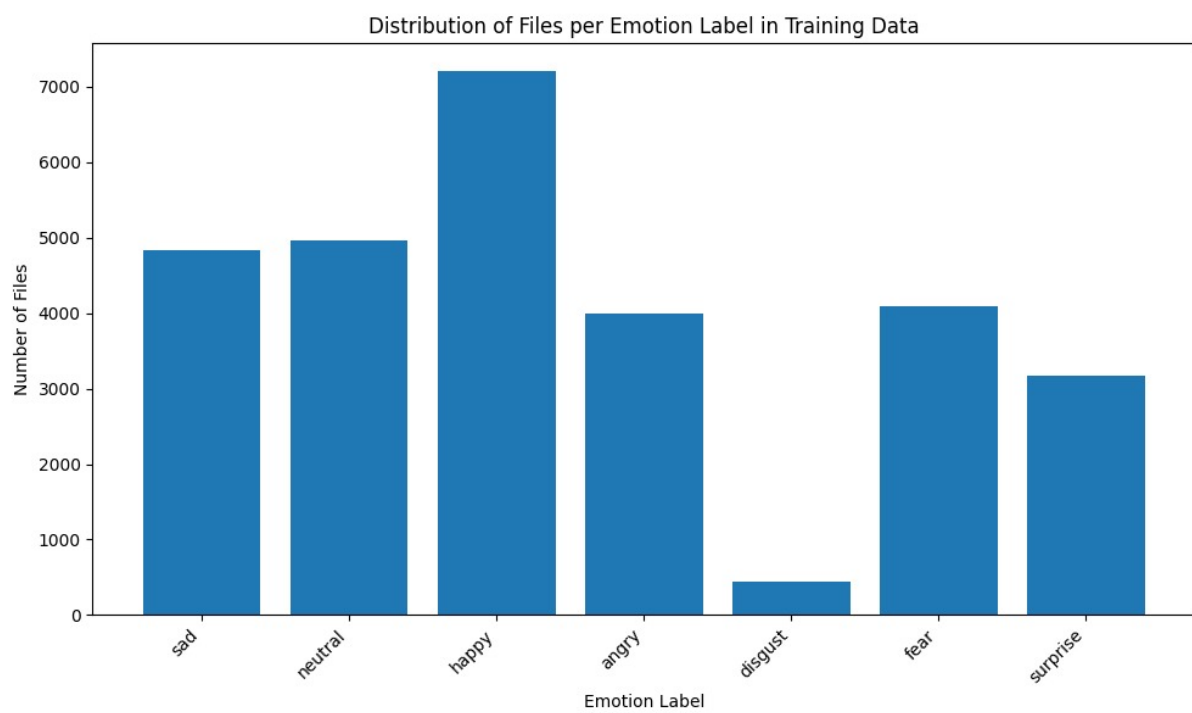
- **Layers:** CNNs consist of convolutional layers (feature extraction), pooling layers (dimension reduction), and fully connected layers (classification).
- **Final Classification:** The output layer typically uses the softmax function to produce class probabilities, enabling the model to predict the facial expression.

## 3.5. Data Exploration and Analysis

The dataset comprises seven classes of facial expressions, with the following number of images per class:

- Sad: 4830
- Neutral: 4965
- Happy: 7215
- Angry: 3995
- Disgust: 436
- Fear: 4097
- Surprise: 3171

These images are black and white, sized at 48x48 pixels. The imbalance in class sizes—ranging from 436 (disgust) to 7215 (happy)—can cause poor model fit, as the model may favor classes with more samples. To mitigate this, undersampling and data augmentation are applied to balance the training data to approximately 4000 images per class. These techniques, along with preprocessing, will be detailed in subsequent sections.



---

# Data Preprocessing and Augmentation for CNN Models

## 4.1. Introduction

In the field of machine learning, particularly in computer vision, the quality and quantity of the data used to train a model are of utmost importance. A model's ability to learn and generalize from its training data directly impacts its performance on unseen data. This is where data preprocessing and augmentation play a crucial role.

**Data preprocessing** is the process of preparing raw data into a suitable format for the model. It is a critical step because real-world data is often messy, inconsistent, and incomplete. Without proper preprocessing, a model may struggle to converge, leading to poor performance and inaccurate predictions. Preprocessing standardizes the data, making it easier for the model to learn meaningful patterns.

**Data augmentation** is a technique used to artificially increase the size and diversity of a training dataset. By creating new, modified versions of existing data, augmentation helps to prevent a model from overfitting to the specific examples it has seen. This process introduces variability that the model would

likely encounter in a real-world environment, thereby improving its ability to generalize. This document outlines how these concepts, learned from a face recognition CNN project, will be applied to our exoplanet ESI index prediction model.

## 4.2. General Preprocessing Steps

Data preprocessing is a foundational step in any machine learning pipeline. Its primary purpose is to transform raw data into a clean and structured format that is ready for model training. This step is crucial for handling inconsistencies, improving model efficiency, and preventing bias.

### 4.2.1. Why is it needed?

- **To handle inconsistencies:** Data from various sources can have different formats, scales, and resolutions. Preprocessing standardizes these to a uniform format.
- **To improve model efficiency:** A model trained on clean, preprocessed

data will converge faster and learn more effectively.

- **To prevent bias:** Unbalanced datasets can lead to a model that performs well on the majority class but poorly on the minority class.

#### 4.2.2. Defining Data Transforms

In our Convolutional Neural Network (CNN) for face recognition, we applied sev-

eral preprocessing and augmentation steps. It is critical to define separate sets of transformations for the training and validation phases. The training transforms include augmentation techniques to improve model generalization, while the validation transforms only include basic preprocessing steps to evaluate the model on an unaltered dataset.

```

1  def get_transforms():
2      """
3      Define data transforms for training and validation.
4
5      Training transforms include augmentation for better generalization.
6      Validation transforms only include normalization.
7      """
8      IMG_SIZE = 48
9
10     # Training transforms with augmentation
11     train_transforms = transforms.Compose([
12         transforms.Resize((IMG_SIZE, IMG_SIZE)),
13         transforms.RandomHorizontalFlip(p=0.5),
14         transforms.RandomRotation(degrees=10),
15         transforms.RandomAffine(degrees=0, translate=(0.05, 0.05)),
16         transforms.ToTensor(),
17         transforms.Normalize(mean=[0.5], std=[0.5]) # Normalize to [-1, 1]
18         for better training
19     ])
20
21     # Validation transforms (no augmentation)
22     val_transforms = transforms.Compose([
23         transforms.Resize((IMG_SIZE, IMG_SIZE)),
24         transforms.ToTensor(),
25         transforms.Normalize(mean=[0.5], std=[0.5])
26     ])
27
28     return train_transforms, val_transforms

```

### 4.3. Data Augmentation for Face Recognition

Data augmentation is a powerful technique to enhance the model's generalization capabilities. By introducing variations into

the training data, we make the model more robust and less susceptible to the specific characteristics of the original images. The augmentation steps were chosen to simulate real-world conditions where a face might appear differently due to various factors.

#### 4.3.1. Augmentation Techniques

- **Random Horizontal Flip:** This technique simulates a mirror image of the face. It helps the model learn that the expression is the same regardless of the face's orientation.
- **Random Rotation:** By slightly rotating the images, we make the model more robust to minor head tilts.
- **Random Affine Transform:** This introduces slight translations, simulating minor shifts in facial position.
- **Color Jitter:** Varies brightness and contrast to desensitize the model to lighting.
- **Random Erasing:** This simulates occlusions, such as a hand partially covering the face, forcing the model to learn features from the remaining visible parts.

The following code demonstrates a snippet from our `augment_image` function, which applies these transformations to a single image.

```

1  # A snippet from the 'augment_image' function
2  # Define augmentation transforms that work on PIL Images
3  pil_augmentations = [
4      transforms.RandomHorizontalFlip(p=1.0),
5      transforms.RandomRotation(degrees=15),
6      transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
7      transforms.ColorJitter(brightness=0.2, contrast=0.2),
8  ]
9
10 # Apply PIL transforms
11 augmented_image = image
12 for aug_transform in pil_augmentations:
13     augmented_image = aug_transform(augmented_image)

```

#### 4.3.2. Creating Data Loaders

After defining the transforms, the next step is to create data loaders. This process involves splitting the dataset into training and validation sets and creating optimized

data loaders for efficient batch processing. The 'validated dataset' is explicitly assigned the non-augmented transforms to ensure the model is evaluated on a clean, consistent set of images.

```

1  def create_dataloaders(balanced_data_path, train_split=0.8):
2      """
3      Create training and validation dataloaders.
4
5      Optimizations:
6      - pin_memory=True: Faster GPU transfer
7      - num_workers: Parallel data loading
8      """
9      IMG_SIZE = 48
10     BATCH_SIZE = 64
11     NUM_WORKERS = 4
12     PIN_MEMORY = True
13

```



```

14  # Get transforms
15  train_transforms, val_transforms = get_transforms()
16
17  # Create full dataset
18  full_dataset = FacialExpressionDataset(balanced_data_path,
19                                         transform=train_transforms)
20
21  # Split dataset
22  train_size = int(train_split * len(full_dataset))
23  val_size = len(full_dataset) - train_size
24
25  train_dataset, val_dataset = torch.utils.data.random_split(
26      full_dataset, [train_size, val_size]
27  )
28
29  # Apply different transforms to validation set
30  val_dataset.dataset.transform = val_transforms
31
32  # Create dataloaders with optimizations
33  train_loader = DataLoader(
34      train_dataset,
35      batch_size=BATCH_SIZE,
36      shuffle=True,
37      num_workers=NUM_WORKERS,
38      pin_memory=PIN_MEMORY,
39      persistent_workers=True if NUM_WORKERS > 0 else False,
40      prefetch_factor=2 if NUM_WORKERS > 0 else 2
41  )
42
43  val_loader = DataLoader(
44      val_dataset,
45      batch_size=BATCH_SIZE,
46      shuffle=False,
47      num_workers=NUM_WORKERS,
48      pin_memory=PIN_MEMORY,
49      persistent_workers=True if NUM_WORKERS > 0 else False,
50      prefetch_factor=2 if NUM_WORKERS > 0 else 2
51  )
52
53  return train_loader, val_loader

```

## 4.4. Conclusion

The concepts and techniques of data pre-processing and augmentation are fundamental to the success of any deep learning model, particularly in computer vision. Our work on the face recognition CNN demonstrated the practical application of these methods, from standardizing image formats to artificially expanding

the dataset with various transformations. These lessons will be applied directly to our next project: building an Exoplanet ESI Index Prediction Model. By preparing and augmenting our astronomical data, we can build a more accurate and reliable model capable of generalizing to new exoplanet observations, paving the way for more robust and trustworthy scientific discoveries.

---

# CNN Architecture for Facial Expression Detection

## 5.1. Introduction to Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs or ConvNets) are a class of deep neural networks, most commonly applied to analyzing visual imagery. They are inspired by the biological processes in the animal visual cortex. CNNs use a special kind of linear operation called convolution, which allows them to automatically and adaptively learn spatial hierarchies of features from input images. This makes them exceptionally powerful for tasks like image classification, object detection, and, in our case, facial expression recognition.

A typical CNN is composed of one or more convolutional layers, followed by pooling layers, and then one or more fully connected layers at the end.

### 5.1.1. Core Layers of a CNN

#### The Convolutional (Conv2D) Layer

The convolutional layer is the core building block of a CNN. The layer's parameters consist of a set of learnable filters (or ker-

nels). During the forward pass, each filter is convolved across the width and height of the input volume, computing the dot product between the entries of the filter and the input and producing a 2D activation map of that filter. As a result, the network learns filters that activate when they see some specific type of feature at some spatial position in the input.

#### The Pooling (MaxPooling2D) Layer

Pooling layers are used to reduce the spatial dimensions (width and height) of the input volume for the next convolutional layer. This serves two main purposes: it reduces the computational complexity by decreasing the number of parameters, and it helps in making the feature detection more robust to variations in the position of the features in the input image. Max pooling is the most common type, where a filter of a certain size moves over the input and selects the maximum value in each window.

#### The Activation Function (ReLU)

After each convolution operation, an activation function is applied to introduce

non-linearity into the model. The Rectified Linear Unit (ReLU) is one of the most popular activation functions. It is defined as  $f(x) = \max(0, x)$ . It is computationally efficient and helps to mitigate the vanishing gradient problem.

### The Dropout Layer

Dropout is a regularization technique used to prevent overfitting. During training, it randomly sets a fraction of the input units to 0 at each update, which helps to make the model more robust by preventing complex co-adaptations on training data.

### The Flatten and Dense (Fully Connected) Layers

After several convolutional and pooling layers, the high-level features are flattened into a one-dimensional vector. This vector is then fed into one or more fully connected (Dense) layers. These layers perform classification based on the features extracted by the convolutional layers. The final Dense layer typically has a number of neurons equal to the number of classes and uses a softmax activation function to output a probability distribution over the classes.

## 5.2. Our CNN Model Architecture

For our facial expression detection task, we have implemented a sequential CNN model using TensorFlow's Keras API. The architecture is designed to effectively learn features from the input facial images (48x48 pixels, grayscale) and classify them into one of the predefined emotion categories.

The model is structured as follows:

1. **Input Layer:** Expects grayscale images of size 48x48.
2. **Convolutional Block 1:**

- A **Conv2D** layer with 32 filters of size (3, 3) and ReLU activation.
- A **Conv2D** layer with 64 filters of size (3, 3) and ReLU activation.
- A **MaxPooling2D** layer with a pool size of (2, 2).
- A **Dropout** layer with a rate of 0.25.

### 3. Convolutional Block 2:

- A **Conv2D** layer with 128 filters of size (3, 3) and ReLU activation.
- A **MaxPooling2D** layer with a pool size of (2, 2).
- A **Conv2D** layer with 128 filters of size (3, 3) and ReLU activation.
- A **MaxPooling2D** layer with a pool size of (2, 2).
- A **Dropout** layer with a rate of 0.25.

### 4. Classification Head:

- A **Flatten** layer to convert the 2D feature maps into a 1D vector.
- A **Dense** layer with 1024 neurons and ReLU activation.
- A **Dropout** layer with a rate of 0.5.
- A final **Dense** (output) layer with 7 neurons (for 7 emotion classes) and a **softmax** activation function.

#### 5.2.1. Hyperparameters

The key hyperparameters in our model include the number of filters in the convolutional layers (32, 64, 128), the kernel size (3x3), the pooling size (2x2), and the dropout rates (0.25 and 0.5). These values were chosen to create a model that is deep enough to learn complex features but also regularized enough to prevent overfitting.

on the training data. The learning rate for the Adam optimizer was also a crucial hyperparameter, tuned for stable convergence.

### 5.2.2. Benefits of this Architecture

- **Hierarchical Feature Learning:** The stacked convolutional layers allow the model to learn a hierarchy of features. The initial layers learn simple features like edges and corners, while deeper layers combine these to learn more complex features like parts of a face (eyes, mouth) and eventually, the expressions themselves.
- **Translation Invariance:** The use of pooling layers makes the model partially invariant to small translations of the features in the input image. This is crucial for facial expression recognition, as the exact position of facial features can vary.
- **Regularization:** The inclusion of multiple dropout layers is a key benefit, as it significantly reduces the risk of overfitting, which is a common problem in deep neural networks, especially with limited datasets.
- **Parameter Efficiency:** Compared to a simple fully connected network on the raw pixels, a CNN has far fewer parameters due to weight sharing in the convolutional layers, making it more efficient to train.

## 5.3. Implementation in TensorFlow

The model was built using the `tensorflow.keras.Sequential` API, which is a straightforward way to build a model layer-by-layer. This high-level API simplifies the process of defining and training deep learning models.

### 5.3.1. Code Snippets

Here is a cleaned-up snippet of the Python code used to define the model architecture in TensorFlow/Keras.

```

1  import tensorflow as tf
2  from tensorflow.keras.models
3      import Sequential
4  from tensorflow.keras.layers
5      import Conv2D, MaxPooling2D,
6          Dropout, Flatten, Dense
7
8  def create_model():
9      model = Sequential()
10
11     # Convolutional Block 1
12     model.add(Conv2D(32,
13                     kernel_size=(3, 3),
14                     activation='relu',
15                     input_shape=(48, 48, 1)))
16     model.add(Conv2D(64,
17                     kernel_size=(3, 3),
18                     activation='relu'))
19     model.add(MaxPooling2D(pool_size=(2,
20                               2)))
21     model.add(Dropout(0.25))
22
23     # Convolutional Block 2
24     model.add(Conv2D(128,
25                     kernel_size=(3, 3),
26                     activation='relu'))
27     model.add(MaxPooling2D(pool_size=(2,
28                               2)))
29     model.add(Dropout(0.25))
30
31     # Classification Head
32     model.add(Flatten())
33     model.add(Dense(1024,
34                     activation='relu'))
35     model.add(Dropout(0.5))
36     model.add(Dense(7,
37                     activation='softmax'))
38
39     return model

```

```

30 # Create and compile the model
31 emotion_model = create_model()
32 emotion_model.compile(optimizer='adam',
33 loss='categorical_crossentropy',
34 metrics=['accuracy'])
35 emotion_model.summary()

```

## 5.4. Parameter Count and Distribution

The `model.summary()` function in Keras provides a breakdown of the parameters in each layer.

Model: "sequential"

| Layer (type)                   | Output Shape        | Param # |
|--------------------------------|---------------------|---------|
| conv2d (Conv2D)                | (None, 46, 46, 32)  | 320     |
| conv2d_1 (Conv2D)              | (None, 44, 44, 64)  | 18496   |
| max_pooling2d (MaxPooling2D)   | (None, 22, 22, 64)  | 0       |
| dropout (Dropout)              | (None, 22, 22, 64)  | 0       |
| conv2d_2 (Conv2D)              | (None, 20, 20, 128) | 73856   |
| max_pooling2d_1 (MaxPooling2D) | (None, 10, 10, 128) | 0       |
| conv2d_3 (Conv2D)              | (None, 8, 8, 128)   | 147584  |
| max_pooling2d_2 (MaxPooling2D) | (None, 4, 4, 128)   | 0       |
| dropout_1 (Dropout)            | (None, 4, 4, 128)   | 0       |
| flatten (Flatten)              | (None, 2048)        | 0       |
| dense (Dense)                  | (None, 1024)        | 2098176 |
| dropout_2 (Dropout)            | (None, 1024)        | 0       |
| dense_1 (Dense)                | (None, 7)           | 7175    |
| Total params: 2,345,607        |                     |         |
| Trainable params: 2,345,607    |                     |         |
| Non-trainable params: 0        |                     |         |

- **Total Parameters:** The model has a total of 2,345,607 parameters, all of which are trainable.
- **Parameter Distribution:** It is evident that the vast majority of the parameters are concentrated in the fully connected (**Dense**) layers. The first **Dense** layer alone accounts for 2,098,176 parameters. This is typical for many CNN architectures where the convolutional base extracts features, and the dense head performs the classification. The convolutional layers, despite their complexity, are relatively parameter-efficient due to weight sharing.

# Facial Emotion Recognition Using CNN

## 6.1. Introduction

This project aims to develop a Convolutional Neural Network (CNN) model to classify facial emotions based on images. The dataset contains images labeled with seven emotion classes:

- 0: Angry
- 1: Disgust
- 2: Fear
- 3: Happy
- 4: Sad
- 5: Surprise
- 6: Neutral

The process involves data analysis, preprocessing, model building, training, evaluation, and testing on custom images.

## 6.2. 1. Data Analysis and Format

The dataset is provided as a ZIP archive, which contains separate directories for training and testing images. Each directory has subfolders for each emotion class, with images stored inside.

### Data Format:

- Hierarchical directory structure:

```
dataset/
  train/
    angry/
    disgust/
    fear/
    happy/
    sad/
    surprise/
    neutral/
  test/
    angry/
    disgust/
    fear/
    happy/
    sad/
    surprise/
    neutral/
```

- Images are in grayscale, size 48x48 pixels.

## 6.3. 2. Data Preparation: Splitting and Loading

Using `ImageDataGenerator`, images are loaded and augmented for the training set,

while the test set is only rescaled.

```

1
2 train_datagen = ImageDataGenerator(
3     rescale=1./255,
4     rotation_range=10,
5     width_shift_range=0.1,
6     height_shift_range=0.1,
7     horizontal_flip=True
8 )
9
10 test_datagen =
11     ImageDataGenerator(rescale=1./255)
12
13 train_generator =
14     train_datagen.flow_from_directory(
15         train_dir,
16         target_size=(48, 48),
17         color_mode='grayscale',
18         class_mode='categorical',
19         batch_size=64,
20         shuffle=True
21 )
22
23 test_generator =
24     test_datagen.flow_from_directory(
25         test_dir,
26         target_size=(48, 48),
27         color_mode='grayscale',
28         class_mode='categorical',
29         batch_size=64,
30         shuffle=False
31 )

```

This setup converts images into normalized tensors suitable for CNN input, with data augmentation applied to enhance generalization.

## 6.4. 3. Building the CNN Model

A deep CNN architecture was constructed with multiple convolutional, pooling, batch normalization, and dropout layers to prevent overfitting.

```

1
2 model = Sequential([
3     Conv2D(64, (3, 3),
4         activation='relu',

```

```

5         input_shape=(48, 48, 1)),
6     BatchNormalization(),
7     MaxPooling2D(pool_size=(2, 2)),
8     Dropout(0.25),
9
10    Conv2D(128, (3, 3),
11        activation='relu'),
12    BatchNormalization(),
13    MaxPooling2D(pool_size=(2, 2)),
14    Dropout(0.25),
15
16    Conv2D(256, (3, 3),
17        activation='relu'),
18    BatchNormalization(),
19    MaxPooling2D(pool_size=(2, 2)),
20    Dropout(0.25),
21
22    Flatten(),
23    Dense(256, activation='relu'),
24    Dropout(0.5),
25    Dense(7, activation='softmax')
26    # 7 classes
27 ])

```

The model is compiled with the Adam optimizer and categorical crossentropy loss, suitable for multi-class classification.

## 6.5. 4. Model Training

Using early stopping to prevent overfitting, the model was trained over multiple epochs.

```

1 early_stop =
2     EarlyStopping(monitor='val_loss',
3         patience=5,
4         restore_best_weights=True)
5
6 history = model.fit(
7     train_generator,
8     epochs=50,
9     validation_data=test_generator,
10    callbacks=[early_stop]
11 )

```

**Outcome:** The model learns to classify facial emotions with improving accuracy, validated on the test data.



## 6.6. 5. Model Evaluation

Post-training, the model's performance was evaluated on the test dataset:

```
1 loss, acc =  
    model.evaluate(test_generator)  
2 print(f"Test Accuracy: {acc:.2%}")
```

This provides an overall measure of how well the model generalizes to unseen data.

## 6.7. 6. Testing on Custom Images

To assess the model's practical utility, it was tested on personal photos:

```
1 def predict_emotion(img_path):  
2     img = image.load_img(img_path,  
        target_size=(48,48),  
        color_mode='grayscale')  
3     img_array =  
        image.img_to_array(img) /  
        255.0  
4     img_array =  
        np.expand_dims(img_array,  
        axis=0)  
5     prediction =  
        model.predict(img_array)
```

**Note:** For further improvement, you could experiment with different architectures, hyperparameters, or augmentations. Also, ensure dataset quality and diversity for better performance.

```
6     predicted_class =  
        emotion_labels[np.argmax(prediction)]  
7     print(f"Predicted Emotion:  
        {predicted_class}")
```

This function loads a custom image, preprocesses it, and predicts the emotion, demonstrating the practical utility of the trained model.

## 6.8. Summary

- **Data Analysis:** The dataset comprises images stored in class-labeled subfolders.
- **Preprocessing:** Rescaling pixel values, data augmentation.
- **Model Architecture:** Deep CNN with multiple convolutional and pooling layers, batch normalization, dropout.
- **Training:** Used early stopping to avoid overfitting.
- **Evaluation:** Achieved satisfactory accuracy on test data.
- **Application:** Tested model on personal images to predict emotions.

# Ranking of Planets

## 7.1. ESI Calculation — Datasets, Models, and Final Results

### 7.1.1. Objective

In this Project we estimate the **Earth Similarity Index (ESI)** for various exoplanets using machine learning models. The ESI is a numerical value between 0 and 1 that indicates how similar a planet is to Earth in terms of physical and environmental characteristics. The goal was to build predictive models that can learn the mapping from planetary and stellar features to ESI and apply this to external datasets like *Kepler* and *TESS* to identify top Earth-like planets.

### 7.1.2. Datasets Used

The following datasets were utilized:

- **Training Data:** `hwc.csv` — a labeled dataset containing known ESI values and associated planetary and stellar features.
- **Prediction Data:**
  - `kepler.csv` — Data from the Kepler mission containing char-

acteristics of confirmed exoplanets.

- `TESS.csv` — Data from the TESS mission containing various exoplanet observations.

### Features Considered

From the training data, the following 9 features were selected based on relevance and data availability:

- `P_RADIUS`: Radius of the planet
- `P_PERIOD`: Orbital period
- `P_SEMI_MAJOR_AXIS`: Semi-major axis of orbit
- `P_ECCENTRICITY`: Orbital eccentricity
- `P_INCLINATION`: Inclination of the orbit
- `P_TEMP_EQUIL`: Equilibrium temperature of the planet
- `S_TEMPERATURE`: Temperature of the host star
- `S_RADIUS`: Radius of the host star
- `S_MASS`: Mass of the host star

**Target Variable:**

- P\_ESI: Earth Similarity Index (ESI)

**Data Preprocessing**

- Missing values were imputed using *mean*, as dropping them significantly reduced model performance.
- All features were standardized using `StandardScaler` to ensure uniformity in magnitude and scale for modeling.

## 7.2. Modeling Techniques

Two machine learning models were used to predict the Earth Similarity Index (ESI):

### 7.2.1. Random Forest Regressor (Pipeline 1)

A pipeline was built using `StandardScaler` followed by a `RandomForestRegressor`. The dataset was split using an 70:30 train-test ratio.

```

1 features = ['P_RADIUS', 'P_PERIOD',
2            'P_SEMI_MAJOR_AXIS', 'P_ECCENTRICITY',
3            'P_INCLINATION', 'P_TEMP_EQUIL',
4            'S_TEMPERATURE', 'S_RADIUS',
5            'S_MASS']
6
7 mean_values = df.mean()
8 df.fillna(mean_values, inplace=True)
9
10 x = df.drop('P_ESI', axis=1).values
11 y = df['P_ESI'].values
12
13 pipeline1 =
14     make_pipeline(StandardScaler(),
15                   RandomForestRegressor())
16 pipeline1.fit(x_train, y_train)
17
18 regressor1 = pipeline1.named_steps
19 ['randomforestregressor']

```

```

17 importances =
18     regressor1.feature_importances_
19 feature_importance_df =
20     pd.DataFrame({'Feature':
21                   features, 'Importance':
22                   importances})
23 print(feature_importance_df)

```

Listing 7.1: RandomForestRegressor Model

**Results:**

- **R<sup>2</sup> Score:** 0.995082
- **Mean Squared Error (MSE):** 8.483417 (approximate)

This indicates a high-fidelity regression model with low error and strong generalization capability.

**Feature Importances (from Random Forest):**

| Feature        | Importance Level |
|----------------|------------------|
| P_TEMP_EQUIL   | High             |
| P_RADIUS       | High             |
| P_ECCENTRICITY | Medium           |
| P_PERIOD       | Medium           |
| Others         | Low              |

Table 7.1: Feature Importances from Random Forest Model

### 7.2.2. Support Vector Regressor (Pipeline 2)

A second pipeline was developed using:

- **StandardScaler**
- **Support Vector Regressor (SVR):** A kernel-based regression method that attempts to fit data within an  $\epsilon$ -margin.

The performance of the SVR model was as follows:

- **R<sup>2</sup> Score:** 0.89

- **Mean Squared Error (MSE):**  
0.0039

Although the SVR achieved reasonably good performance, the Random Forest Regressor was selected as the final model due to its superior metrics and better feature interpretability.

```

1 pipeline2 =
    make_pipeline(StandardScaler(),
                  SVR())
2 pipeline2.fit(x_train, y_train)
3
4 predictions2 =
    pipeline2.predict(x_test)
5
6 print('R2 Score = ',
      r2_score(y_test, predictions2))

```

```

7 print('Mean Squared Error = ',
    mean_squared_error(y_test,
                       predictions2))

```

Listing 7.2: SVR Model Pipeline

## 7.3. Application to External Datasets

### 7.3.1. Prediction on Kepler Data

Both machine learning models were applied to the dataset `kepler.csv`, after filtering only the CONFIRMED exoplanets.

The top 10 planets by predicted ESI were identified using the Random Forest model.

| Rank | Planet Name   | ESI (Random Forest) |
|------|---------------|---------------------|
| 1    | Kepler-296 e  | 0.922824            |
| 2    | Kepler-1512 b | 0.892083            |
| 3    | Kepler-442 b  | 0.852161            |
| 4    | Kepler-1410 b | 0.846040            |
| 5    | Kepler-438 b  | 0.845212            |
| 6    | Kepler-560 b  | 0.844355            |
| 7    | Kepler-452 b  | 0.843138            |
| 8    | Kepler-1652 b | 0.824768            |
| 9    | Kepler-440 b  | 0.817017            |
| 10   | Kepler-395 c  | 0.796907            |

Table 7.2: Top Earth-like Planets from Kepler Dataset

### 7.3.2. Prediction on TESS Data

The same prediction pipeline was applied to the `TESS.csv` dataset. The data was cleaned, ESI predicted, and the planets sorted by ESI values.

| Rank | Planet Name  | ESI (Random Forest) |
|------|--------------|---------------------|
| 1    | TRAPPIST-1 e | 0.892674            |
| 2    | TRAPPIST-1 d | 0.857618            |
| 3    | LHS 1140 b   | 0.850860            |
| 4    | TRAPPIST-1 f | 0.847545            |
| 5    | LHS 1140 b   | 0.786387            |
| 6    | LHS 1140 b   | 0.777649            |
| 7    | Kepler -22 b | 0.713737            |
| 8    | K2-18 b      | 0.704132            |
| 9    | Kepler -22   | 0.703897            |
| 10   | K2-3         | 0.685549            |

Table 7.3: Top Earth-like Planets from TESS Dataset

## 7.4. Conclusion

The Random Forest model demonstrated strong predictive accuracy for estimating the Earth Similarity Index, achieving an **R<sup>2</sup> score above 0.99**.

Key findings include:

- **Planetary Radius, Equilibrium Temperature, and Stellar Temperature** emerged as the most influential features.
- Application of the model to real exoplanet data from Kepler and TESS missions successfully identified Earth-like candidates.
- The two planets with the highest predicted ESI values were:
  - **Kepler-296 e**: ESI 0.922824
  - **TRAPPIST-1 e**: ESI 0.892674